

Rfacto

Architecture — appels audio et vidéo 1-to-1

Document interne | 16 août 2026 | analyse uniquement, aucun code

Proposition d'intégration d'appels audio et vidéo 1-to-1 de type réseau social, calée sur le code existant (Android rfactocan + API Next.js / Neon). Aucun fichier n'a été modifié pour cette analyse.

1. Constat — ce qui existe déjà

Rfacto n'a ni WebSocket, ni WebRTC, ni permission micro/caméra. La messagerie et le push sont HTTP + FCM data-only. C'est un bon plan de contrôle (qui appelle qui, sonnerie, historique), pas un plan média pour transporter l'audio/vidéo.

Architecture Android

- Une seule Activity : MainActivity.kt — navigation par currentScreen (app, dm_chat, chat...), pas de NavHost Jetpack.
- Shell : MainShell.kt + onglets AppTab (dont Messages).
- Réseau : Retrofit + OkHttp, Bearer JWT (RetrofitClient.kt).
- Session : SessionManager (SharedPreferences rfacto_prefs).
- Pas de classe Application, pas de ForegroundService, pas de ConnectionService.
- minSdk 24, targetSdk 36, package com.rfacto.app.

Messagerie (deux silos)

Fil	Tables / API	Android	Temps réel
DM (services, emploi, rencontre)	DirectThread + DirectMessage	DirectChatScreen.kt	poll 4 s
Colis / réservation	Message sur booking	ChatScreen.kt	poll 8 s

Règles DM (lib/dm.ts, app/api/dm/route.ts) : couple unique userLowId / userHighId ; KYC VERIFIED des deux côtés (sauf MEET) ; MEET : MeetContact ACCEPTED (pas de cold DM) ; compte SUSPENDED refusé. C'est le bon périmètre pour autoriser un appel : uniquement entre participants d'un DirectThread existant.

Auth, backend, présence, FCM

- JWT (lib/auth.ts) : cookie web ou Authorization Bearer mobile.
- Android enregistre le jeton FCM après login (PushRegistrar → POST /api/devices/fcm).
- Next.js App Router sur Vercel (fonctions serverless, pas de socket persistant).
- Prisma / Neon PostgreSQL. FCM Admin : lib/fcm.ts + notifyUser.
- Présence : POST /api/presence (lastSeenAt toutes les 45 s). Seuil online : 2 min.
- FCM data-only Android, priority high. Canaux rfacto_messages_v3 / alerts_v3 / jobs_v3.
- RfactoFirebaseMessagingService construit la notif et ouvre MainActivity via extras. Pas de USE_FULL_SCREEN_INTENT, pas d'écran d'appel.

WebSocket

Aucun. Ni Socket.IO, ni ws, ni Pusher. Supabase est dans le repo (lib/supabase) mais n'est pas utilisé pour du realtime.

Permissions actuelles

INTERNET, localisation, POST_NOTIFICATIONS, VIBRATE. Absentes : RECORD_AUDIO, CAMERA, FOREGROUND_SERVICE*, USE_FULL_SCREEN_INTENT, BLUETOOTH_CONNECT.

Cycle de vie

Splash → login → currentScreen = app. Presence seulement si l'écran est app ou chat. Processus tué : seul FCM réveille l'app (déjà le cas pour les messages).

2. Contrainte décisive

Vercel ne peut pas héberger le signaling WebRTC (WebSocket longue durée, ICE trickle, TURN). Un WS maison sur Next.js cassera en production.

Il faut séparer :

- Plan de contrôle Rfacto (déjà là) : JWT, thread, KYC, FCM, historique Neon.
- Plan média (nouveau service) : WebRTC + STUN/TURN + signaling.

3. Architecture recommandée

LiveKit Cloud (ou Daily.co) comme plan média, Rfacto comme plan de contrôle.

Pourquoi LiveKit plutôt qu'un WS + libwebrtc maison :

- TURN/STUN et SFU déjà opérés (Afrique / NAT mobile, Motorola).
- SDK Android + Web.
- Signaling WS vers leurs serveurs, pas Vercel.
- Identité Rfacto (JWT) pour créer l'appel et sonner.

Appelant → POST /api/calls → Rfacto crée Call + room LiveKit + FCM INCOMING_CALL → Callee IncomingCallActivity → accept → token LiveKit → join room WebRTC.

FCM sonne. LiveKit transporte. Neon historise. Ne pas utiliser le poll 4 s du chat pour l'appel : trop lent. Le ring passe par FCM ; pendant l'appel, le SDK LiveKit gère ICE/SDP.

4. Les 11 points, calés sur Rfacto

#	Besoin	Comment dans ce projet
1	Audio 1-to-1	Room LiveKit audio=true, video=false. Bouton dans DirectChatScreen.
2	Vidéo 1-to-1	Même room, video=true. Même Call avec mediaType.
3	WebRTC	SDK LiveKit (enveloppe WebRTC). Pas de PeerConnection à la main au début.
4	Signaling WS	WS LiveKit, pas un serveur Rfacto. Optionnel : GET /api/calls/:id pour l'état.
5	TURN/STUN	Inclus LiveKit Cloud. coturn seulement si self-host.
6	FCM incoming	notifyUser type=INCOMING_CALL + callId, threadId, mediaType. Canal rfacto_calls_v1.
7	Accept / refus	POST .../accept et .../reject. Timeout 30-45 s → MISSED.
8	Premier plan	onMessageReceived déjà appelé. Overlay IncomingCallScreen via PushEvents.
9	Arrière-plan	Data-only high : le service FCM tourne. Notif full-screen + actions.
10	App fermée	IncomingCallActivity dédiée (pas seulement currentScreen). Full-screen intent Play.
11	Historique	Table Call liée à DirectThread. GET /api/calls. Pastille optionnelle dans le DM.

5. Fichiers concernés (à étendre plus tard)

Android

- MainActivity.kt — extras callId, écrans incoming_call / in_call
- DirectChatScreen.kt — boutons audio/vidéo
- MessagesScreen.kt — accès historique
- RfactoFirebaseMessagingService.kt — type INCOMING_CALL
- JobAlertChannels.kt — canal appels

- PushEvents.kt — event appel
- AndroidManifest.xml — permissions, activity, FGS
- ApiService.kt / RetrofitClient.kt
- Nouveaux : IncomingCallActivity, CallService (foreground), CallViewModel, InCallScreen

Web

- lib/dm.ts — assertThreadParticipant, KYC, MEET
- lib/fcm.ts / lib/notifications.ts
- app/api/dm/**
- Nouveaux : app/api/calls/route.ts, app/api/calls/[id]/accept|reject|token
- prisma/schema.prisma + SQL Neon
- UI web chat si appels aussi sur le site

6. Technologies à réutiliser

- JWT Bearer + getSessionUser
- DirectThread comme graphe « qui peut appeler qui »
- assertBothVerified + MeetContact ACCEPTED
- DeviceToken + FCM data-only high
- notifyUser
- lastSeenAt (online / au téléphone)
- Canal notif + NotificationAlert (sonnerie d'appel dédiée)
- PushEvents pour overlay foreground

7. Nouvelles dépendances

Android

- io.livekit:livekit-android (+ UI Compose LiveKit)
- Permissions : RECORD_AUDIO, CAMERA, MODIFY_AUDIO_SETTINGS, BLUETOOTH_CONNECT, FOREGROUND_SERVICE, FOREGROUND_SERVICE_MICROPHONE, FOREGROUND_SERVICE_CAMERA, USE_FULL_SCREEN_INTENT (API 34+)

Web / infra

- livekit-server-sdk (mint JWT room, serveur only)
- Optionnel web : livekit-client
- Compte LiveKit Cloud (URL + API key/secret dans Vercel)
- Pas de nouveau serveur WS Rfacto au départ

Alternative si refus du SaaS média : serveur Node (Fly/Railway) ws + mediasoup/livekit self-host + coturn — beaucoup plus lourd.

8. Nouvelles tables / API

Call	:	id,	threadId,	callerId,	calleeId,	mediaType	AUDIO VIDEO,	status
RINGING ACCEPTED REJECTED MISSED CANCELED ENDED FAILED, livekitRoom, startedAt, answeredAt, endedAt, endReason. Index : (calleId, status), (threadId, createdAt), (callerId, createdAt).								

Routes (toutes JWT + participant du thread) :

- POST /api/calls { threadId, mediaType }
- GET /api/calls historique

- GET /api/calls/:id état (appellant qui attend)
- POST /api/calls/:id/accept → { livekitUrl, token }
- POST /api/calls/:id/reject | cancel | end
- GET /api/calls/:id/token rejoindre si ACCEPTED

FCM data : type=INCOMING_CALL, callId, threadId, mediaType, callerId, callerName, channelId=rfacto_calls_v1. Cron (comme expire-payments) : RINGING > 45 s → MISSED.

9. Risques techniques

- Vercel ≠ signaling — ne pas coller un WS sur une route Next.
- Android 14/16 full-screen : Play exige une déclaration USE_FULL_SCREEN_INTENT ; sans ça, appel fermé = simple notif, pas d'écran type WhatsApp.
- Foreground service types micro/caméra obligatoires pendant l'appel.
- OEM Motorola : même limite que les notifs (silence, batterie). Data-only high reste le seul réveil.
- KYC / MEET : un bouton d'appel sans les mêmes gardes que le DM = trou sécu.
- Deux silos chat : n'activer les appels que sur DM d'abord, pas sur le chat booking.
- Navigation single-Activity : un appel à froid doit ouvrir une Activity dédiée.
- Coût LiveKit (minutes média) ; 1-to-1 reste raisonnable.
- Demander micro (et caméra si vidéo) avant POST /api/calls, pas après la sonnerie.
- Double son : aujourd'hui FCM joue déjà NotificationAlert.play() ; le canal d'appel ne doit pas doubler une loop + play() one-shot.

10. Plan d'implémentation

- Étape 0 — Produit : appels uniquement depuis un DM existant. Audio d'abord, vidéo ensuite. Pas de cold call.
- Étape 1 — Données + API contrôle : table Call + create/accept/reject/end/list. Pas de média encore.
- Étape 2 — FCM ring : type INCOMING_CALL, canal rfacto_calls_v1. Overlay foreground. Tester Motorola.
- Étape 3 — IncomingCallActivity : Accepter / Refuser indépendant de MainActivity. Timeout.
- Étape 4 — LiveKit audio : mint token, join room, FGS micro.
- Étape 5 — Vidéo : caméra + FGS camera. Même Call.
- Étape 6 — Historique UX : liste Messages, pastille DM, notif MISSED_CALL.
- Étape 7 — Web optionnel : livekit-client sur la page DM.
- Étape 8 — Durcissement : un seul RINGING/ACCEPTED par user ; cron missed ; métriques.

11. Synthèse

Rfacto a déjà identité, fil 1-to-1, KYC, FCM data-only et présence. Il manque le média et un écran d'appel hors MainActivity.

Ne pas construire un signaling WebSocket sur Vercel. Rfacto = qui sonne / qui accepte / historique. LiveKit = WebRTC + TURN + WS média.

Ordre le plus sûr : API Call → FCM ring → IncomingCallActivity → audio LiveKit → vidéo.